ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 13

Service Worker

Рассмотрим технологию Service Worker, позволяющую создавать офлайн-приложения, кэшировать ресурсы и перехватывать сетевые запросы. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Введение

`Service Worker` - это скрипт, который браузер запускает в фоновом режиме, отдельно от веб-страницы. Он открывает возможности, которые не требуют взаимодействия с пользователем или наличия сети: кэширование файлов, push-уведомления, фоновая синхронизация. Именно Service Worker является ключевым компонентом прогрессивных веб-приложений (PWA), с которыми мы ознакомимся несколько позднее.

Что нужно настроить, чтобы началь работу с Service Worker?

Многие функции Service Worker включены по умолчанию в браузерах, поддерживающих эту технологию. Однако, если вы обнаружите, что код не работает в вашей версии браузера, вам может понадобиться их включить:

- Firefox Nightly. Перейдите в раздел about:config и установите параметр dom.serviceWorkers.enabled в значение true; затем перезапустите браузер.
- Chrome Canary. Перейдите в раздел chrome://flags и включите experimental-web-platform-features; перезапустите браузер (заметьте, что некоторые функции теперь включены по умолчанию в браузере Chrome).
- Opera. Перейдите в раздел opera://flags и включите Support for ServiceWorker; перезапустите браузер.

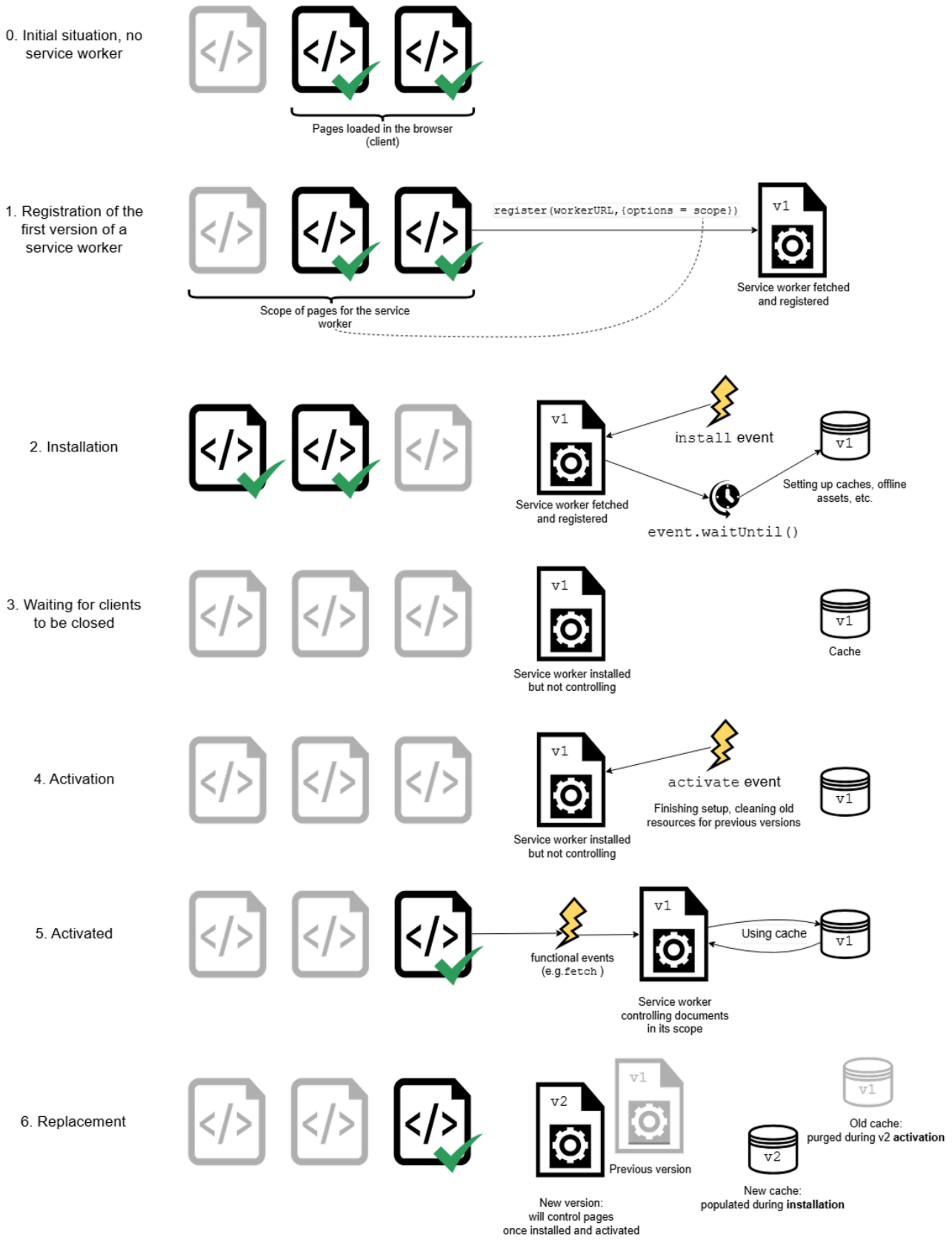
- Microsoft Edge. Перейдите в раздел about:flags и поставьте галочку Enable service workers; перезапустите браузер.

Базовая архитектура

Чтобы сделать базовую настройку Service Worker, нужно пройти несколько этапов:

1. **Регистрация** – вызываем `navigator.serviceWorker.register()`, браузер загружает указанный скрипт.
2. **Запуск в фоне** – после успешной загрузки Service Worker начинает работать в отдельном потоке (`ServiceWorkerGlobalScope`) без доступа к DOM.
3. Теперь Service Worker может обрабатывать свои события.
4. **Установка (событие `install`)** – здесь кэшируем нужные ресурсы (HTML, CSS, JS и т.д.), чтобы приложение могло работать офлайн. Этот этап похож на установку программы – все необходимые файлы сохраняются локально.
5. **Завершение установки** – когда обработчик события `install` завершит свою работу, сервис-воркер считается установленным.
6. **Активация (событие `activate`)** – используется для очистки старых кэшей (например, после обновления скрипта).
7. **Управление страницами** – Service Worker начинает контролировать только те страницы, которые открыты **после** его успешной регистрации. Чтобы применить его к уже открытой вкладке, её нужно перезагрузить.

Наглядное представление описанных этапов представлено на Рисунке ниже.



События Service Worker

Service Worker работает на основе событий. Основные из них делятся на две группы: события жизненного цикла и функциональные события.

События жизненного цикла

- `install` – происходит при первой установке или обновлении Service Worker. Используется для предварительного кэширования ресурсов.
- `activate` – наступает после успешной установки, когда новый Service Worker готов активироваться. Обычно здесь удаляют старые кэши.
- `message` – позволяет получать сообщения от других контекстов (например, от веб-страницы) через `postMessage`.

Функциональные события (возникают при взаимодействии с сетью, фоном и т.д.)

- `fetch` – срабатывает при каждом сетевом запросе из области видимости Service Worker. Даёт возможность перехватить запрос и вернуть ответ из кэша или из сети.
- `sync` – событие фоновой синхронизации. Позволяет отложить действие до появления соединения с сетью (например, отправку данных, добавленных в офлайне).
- `push` – возникает при получении push-уведомления от сервера. Используется для отображения уведомлений, даже когда приложение закрыто.

Реализация простого приложения с офлайн-доступом

Рассмотрим базовую реализацию менеджера заметок, который сохраняет данные в `localStorage` и позволяет просматривать заметки даже без интернета. Service Worker будет кэшировать статические ресурсы, а заметки будут храниться локально, поэтому интерфейс останется рабочим в офлайне. Структура проекта:

```
/notes-app
  ├── index.html
  ├── app.js
  └── sw.js
```

index.html (основная разметка)

```html
<!DOCTYPE html>
<html lang="ru">

<head>
    <meta charset="UTF-8">
    <title>Заметки (13-18)</title>
    <link rel="stylesheet" href="https://unpkg.com/chota@latest">
</head>

<body>
    <h1>А вот и «Заметки» - наше первое оффлайн-приложение</h1>
    <form id="note-form" class="row">
            <input class="col-9" type="text" id="note-input"
placeholder="Введите заметку" required>
            <button class="col-3" type="submit">Добавить</button>
```

```
    </form>
    <ul id="notes-list"></ul>

    <script src="app.js"></script>
</body>

</html>
```

app.js (работа с заметками и регистрация SW)

```
// Элементы DOM
const form = document.getElementById('note-form');
const input = document.getElementById('note-input');
const list = document.getElementById('notes-list');

// Загрузка заметок из localStorage при старте
function loadNotes() {
    const notes = JSON.parse(localStorage.getItem('notes') || '[]');
    list.innerHTML = notes.map(note => `<li>${note}</li>`).join('');
}

// Сохранение заметки
function addNote(text) {
    const notes = JSON.parse(localStorage.getItem('notes') || '[]');
    notes.push(text);
    localStorage.setItem('notes', JSON.stringify(notes));
    loadNotes();
}

// Обработка отправки формы
form.addEventListener('submit', (e) => {
    e.preventDefault();
    const text = input.value.trim();
    if (text) {
        addNote(text);
        input.value = '';
    }
});

// Первоначальная загрузка
loadNotes();

// Регистрация Service Worker
if ('serviceWorker' in navigator) {
    window.addEventListener('load', async () => {
        try {
            const registration = await
navigator.serviceWorker.register('/sw.js');
            console.log('ServiceWorker зарегистрирован:',
```

```
registration.scope);
        } catch (err) {
            console.error('Ошибка регистрации ServiceWorker:', err);
        }
    });
}
```

sw.js (кэшируем все статические файлы)

```
const CACHE_NAME = 'notes-cache-v1';
const ASSETS = [
    '/',
    '/index.html',
    '/app.js'
];

self.addEventListener('install', event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => cache.addAll(ASSETS))
            .then(() => self.skipWaiting())
    );
});

self.addEventListener('activate', event => {
    event.waitUntil(
        caches.keys().then(keys => {
            return Promise.all(
                keys.filter(key => key !== CACHE_NAME)
                    .map(key => caches.delete(key))
            );
        })
    );
});

self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => response || fetch(event.request))
    );
});
```

После запуска сервера (например, через `live-server`) и открытия страницы, в DevTools на вкладке **Application -> Service Workers** можно увидеть активный Service Worker, а в **Cache Storage** – закэшированные файлы. Если отключить сеть и обновить страницу, она всё равно загрузится из кэша. Так приложение становится полностью функциональным без доступа к интернету.

Практическое задание

Необходимо разработать веб-приложение для управления списком дел (или заметок), которое использует Service Worker для обеспечения базового офлайн-функционала. Приложение должно соответствовать следующим требованиям:

- Пользователь может просматривать список задач/заметок.
- Пользователь может добавлять новые задачи/заметки.
- Данные должны сохраняться в `localStorage`.
- Приложение должно регистрировать Service Worker и кэшировать статические ресурсы.
- При отсутствии сети страница должна загружаться из кэша, а интерфейс должен оставаться работоспособным (просмотр и добавление задач должно работать офлайн).

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.

Литература

1. [Service Workers. Инструкция по применению] (https://habr.com/ru/companies/2gis/articles/345552/)